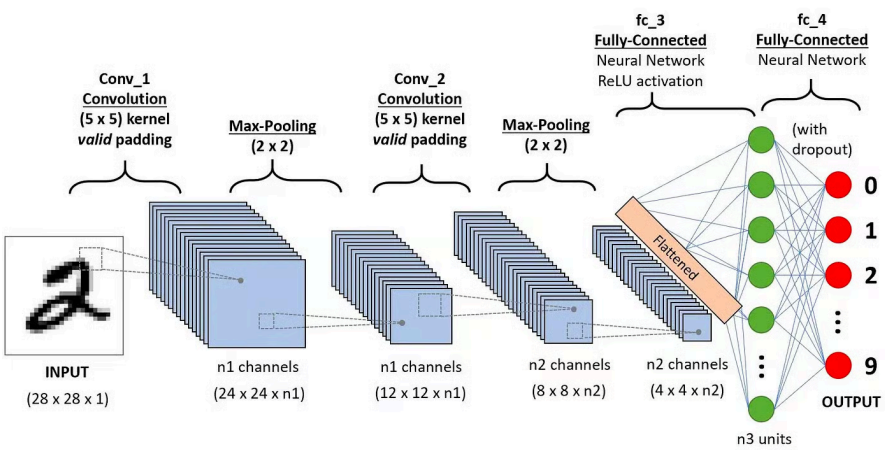
CS148a Assignment 2: Building a Convolutional Neural Network

In this project, you will design, train, and analyze a convolutional neural network (CNN) for MNIST-in-the-wild. You will be evaluated on your understanding of your model's architecture and your training schema. There will be a competition component, with bonus points awarded to the top performing CNNs submitted. The training data will be those collected by yourself and your peers. The test data will not be made public, but is in-domain with the training data.

Task Overview

Using the training data collected by yourself and your classmates, build a CNN that classifies images of digits (0–9) into their correct labels. You are free to choose the depth, width, and architectural details of your model. Materials from lecture, homework, as well as this guide will help you get started. Further instructions are in this **starter colab notebook**⬅️⬅️⬅️. (This notebook has also been uploaded as a .ipynb to the Piazza resources. Feel free to download it there and upload it to Colab or GCP.)



Model Training

You will need a GPU to train your model. If you have your own – great! Otherwise, there are a few ways you can get compute:

1. We are providing $50 GPU credits to each of you using Google Cloud. ☁️***Follow the instructions here***☁️. Make sure to use a Non-Caltech gmail for this.
2. Note that Google also has free Colab Pro for students. To access GPUs that way: go to this page with the Google account you plan to use, choose Pro for students.

Here is a starter notebook ⬅️⬅️⬅️ to get you situated. It contains instructions for loading in the training data, as well as skeleton code for building your CNN and evaluating it. Feel free to run it in Google Colab directly (after making a copy to your own drive). To run it on Google Cloud or on a local machine, simply download the .ipynb notebook.

Required Writeup Questions

In your writeup, answer the following:

Part 1: Training Basics

• How did you train your model? What loss function did you use, and why?
• What learning rate and batch size did you choose? How sensitive was training to these choices? Show any figures you have.
• How long did you train for? Why did you choose that number of epochs? Show any figures you have.
• How deep is your model, and how many kernels are used at each layer?
• How does your model perform during validation? Why do you believe your validation strategy is reliable?
• Why did you make these architectural and optimization choices overall?

Part 2: Data Loading and Preprocessing

• How does your dataloader work? How did you split your training and validation sets?
• What preprocessing or data augmentation steps are applied to the images?
• Why are these processing steps appropriate for this task?
• Which invariances (if any) are you encouraging through your data pipeline?

Part 3: Model Inspection and Interpretation

• Include the output of `print(model)` in your writeup. For each line or block in the printed output, explain what operation it performs.
• Identify which parts of the model are responsible for classification.
• Identify which layers capture low-level visual features and which capture higher-level structure, and explain why.

Part 4: Model Capacity and Parameters

• How many total parameters does your model have?
• Which components contribute most to the parameter count?
• Could the model reasonably have been smaller? Larger?
• What tradeoffs motivated your final model size?

Use of AI Tools

If you use AI in this project, you must clearly document how they were used and include relevant conversation logs in your writeup, under an "AI usage" heading.

Student Deliverables

- **Model Pipeline (.txt)**: Follow instructions at the end of the **starter notebook** ⬅️⬅️⬅️. Upload your model pipeline to Huggingface, then submit the exported json to Gradescope.
- **Writeup (PDF)**: Single PDF (up to 6 pages) addressing all questions above, with figures/tables as needed. *ALSO INCLUDE A LINK TO YOUR FINAL NOTEBOOK*, uploaded to Google Drive with view access enabled for all.

Released: 2/4/2026
Due Date: 2/17/2026

This Version: 02042026_11:53AM